ilo: Architecture (MVP)

Version: 0.1 (Draft for approval) | **Date:** 20 Oct 2026

Source of truth: PRD.md v0.1 and the ilo project context. **Audience:** Coding agent and product owner.

Conventions

- **[AA#]** marks an architecture assumption. **[AD#]** marks a decision that needs your approval before the agent builds that part. All [AD#] items are listed in Section 19.
- PRD references (FR-x, NFR-x, D#, A#) point to PRD.md.
- MVP means required for the pilot. Later means designed for but not built.
- This document contains no application code. Structure, names, and rules only.

1. Architecture Principles

1. **One deployable app plus one managed backend.** A Next.js app and a Supabase project. No microservices, queues, containers, or self-managed servers.
2. **The database enforces access.** Row-Level Security (RLS) is the primary security boundary (NFR-3). The UI and server code are a second layer, never the only one.
3. **The user's identity travels with every query.** The app queries the database as the signed-in user. The elevated service key is used only in a few named server-only places.
4. **Boring and typed.** TypeScript end to end, generated database types, schema validation at every boundary.
5. **Sensitive content stays inside the database.** Emails, logs, analytics, and error reports never carry message text, notes, or check-in content (FR-17, FR-43, NFR-17).
6. **Build only what the PRD lists.** Anything else waits for approval.

2. Recommended Tech Stack

Layer	Choice	Why
Language	**TypeScript** (strict mode)	One language across UI, server, and tests. Generated DB types catch mismatches at build time.

Layer	Choice	Why
Framework	**Next.js (App Router)**	Already chosen in project notes (NFR-16). Server Components, Server Actions, and Route Handlers cover UI, mutations, and webhooks without a separate backend service.
Styling/UI	**Tailwind CSS** with a small set of accessible primitives (shadcn/ui-style components copied into the repo)	Mobile-first (NFR-12), accessible by default (NFR-13), and no heavy runtime component library.
Database	**Supabase Postgres**	Relational data with strong constraints, RLS, and one managed vendor. Already chosen.
Auth	**Supabase Auth** (email OTP / magic link)	Meets FR-1 with no custom auth code. Sessions managed via secure cookies.
Realtime	**Supabase Realtime** (messages and alerts only)	Live message updates with RLS respected. No separate websocket server.
Validation	**Zod**	Single source of truth for input shapes on every Server Action and Route Handler.
Email	**Transactional email provider** (e.g., Resend or equivalent) [AA1]	One outbound integration (PRD Section 11). Provider is swappable behind a thin internal interface.
Scheduled jobs	**Vercel Cron calling protected Route Handlers** [AD1]	Simplest way to send reminders and retry failed emails. No extra infrastructure.
Hosting	**Vercel** (app) and **Supabase** (data)	Zero server management. Preview deployments per branch.
Error monitoring	**Sentry with strict data scrubbing** [AD2]	NFR-17. A single extra dependency, justified by the need to see production failures.

Layer	Choice	Why
Testing	**Vitest** (unit), **SQL-level RLS tests**, **Playwright** (end-to-end)	Covers logic, the security boundary, and critical journeys.
CI/CD	**GitHub Actions**	Lint, type-check, test, migration check on every pull request.

Deliberately not included

- No ORM (the Supabase client plus generated types is enough).
- No separate API server, GraphQL, tRPC, or message queue.
- No Redis or cache layer.
- No state-management library (server data plus local component state).
- No AI/LLM service (see Section 11).
- No container orchestration.

Reasoning summary

This stack lets one developer ship and operate the product. Postgres constraints and RLS carry the correctness and privacy load, so there is little custom backend logic to get wrong.

3. System Overview

3.1 Components

1. **Browser** (students, counsellors, admins), mobile-first web app.
2. **Next.js app on Vercel**
   - Server Components: render pages and read data.
   - Server Actions: all user-initiated writes.
   - Route Handlers: auth callback, cron jobs, exports, health check.
   - Middleware: session refresh and route protection.
3. **Supabase project**
   - Postgres (data, RLS, constraints, functions)
   - Auth (identities and sessions)
   - Realtime (message and alert change streams)

4. **Email provider** (outbound only)
5. **Sentry** (error reports, scrubbed)

3.2 Trust zones

Zone	Contains	Trust level
Browser	UI, user session cookie	Untrusted. Never holds the service key.
Next.js server	Server Actions, Route Handlers	Trusted code, but queries run as the user by default.
Server-only privileged paths	Invitations, cron, notification sending, aggregate analytics	Use the service key; kept in one folder and reviewed carefully (Section 10).
Supabase Postgres	All data	Final authority via RLS and constraints.
Email provider and Sentry	Third parties	Receive metadata only, never sensitive content.

4. Frontend Architecture

4.1 Approach

- **Server-first rendering.** Pages are Server Components that fetch data directly. Client Components are used only for interactivity: booking slot picker, message composer and live thread, mood check-in form, crisis alert button, and the admin charts.
- **Route groups by role.** Student, counsellor, admin, and platform-admin areas are separate route groups, each with its own layout and a server-side role check.
- **Mutations through Server Actions.** Forms submit to a Server Action, which validates, authorises, writes, and returns a typed result. The UI displays success or a friendly error.
- **Persistent crisis control.** The student layout always renders the Need help now? control and the Crisis page with **static fallback helplines** that do not depend on the database (FR-25, NFR-10, PRD Section 14.3).

- **Every screen implements loading, empty, and error states** (PRD Section 14), using route-level loading and error boundaries plus skeleton components.

4.2 Screen-to-route map

PRD screens	Route area
P1-P3	Public routes
S1-S11	Student area
C1-C8	Counsellor area
A1-A4	Institution admin area
PA1-PA2	Platform admin area
403, 404, session-expired, maintenance	Shared system pages

4.3 Client state and realtime

- Server data is the source of truth. Client Components receive initial data from the server.
- **Messages:** a live subscription updates an open thread. If the subscription fails, the UI falls back to periodic refetch and shows no error to the user unless sending fails. [AD3]
- **Unsent text is preserved** on send failure (PRD Section 14).

4.4 Accessibility and responsiveness

- Mobile-first layouts, semantic HTML, visible focus, labelled inputs, sufficient contrast (NFR-12, NFR-13).
- Brand tokens (teal-seafoam default) live in one theme configuration file so the final ilo brand kit can be swapped in without touching components.

5. Backend Architecture

There is no separate backend service. Backend is three things inside the same repo:

1. **Database layer (Postgres):** tables, constraints, RLS policies, and a small set of SQL functions for operations that must be atomic or must bypass row visibility safely (booking, crisis alert creation, aggregate analytics).

2. **Server layer (Next.js):** Server Actions and Route Handlers that validate input, check permissions, call the database, and trigger notifications.
3. **Scheduled layer:** two cron-triggered Route Handlers (reminders, notification retry).

5.1 Layering inside the server code

Layer	Responsibility	Rule
Action / Route Handler	Entry point. Parses input, calls a service, returns a typed result.	No business logic, no direct SQL.
Validation (schemas)	Describes allowed input for each action.	Every action has one; unknown fields are rejected.
Service functions (by domain)	Business rules: booking, messaging, check-ins, crisis, consent, notes, admin.	Only place where rules like cancellation cutoff live.
Data access (repositories)	The only code that talks to the database client.	Always use the user-scoped client unless in the privileged folder.
Notification module	Creates outbox entries and sends email.	Accepts IDs and template names, never free-text sensitive content.

5.2 Operations that must run inside the database

These are implemented as database functions so they are atomic and cannot be bypassed from the app:

- **Book appointment:** checks slot availability and prevents double-booking, in one transaction (FR-11). A uniqueness/exclusion constraint on counsellor and time is the final guard.
- **Create crisis event and notification:** writes the event and the outbox entry together (FR-27, FR-28).
- **Aggregate analytics:** returns only aggregates, applies the fewer-than-5 suppression rule (FR-36, FR-37), and exposes nothing row-level to admins.
- **Account deletion/export preparation:** assembles a user's data and applies the retention exception (FR-6, FR-7, D3).

6. Data Flow

6.1 Sign-in

1. User enters institutional email.
2. Server checks the domain against registered institutions. An unknown domain returns a friendly error without revealing which other domains exist.
3. Supabase Auth emails an OTP/magic link.
4. The callback route exchanges the code for a session cookie.
5. Middleware loads the user's profile and role and routes them to the correct area.
6. A first-time student is sent to consent and onboarding (S1) before anything else is reachable.

6.2 Booking an appointment

1. Student opens a counsellor profile. Server reads availability and existing bookings as the student.
2. Student picks a slot, and the Server Action validates input.
3. The action calls the atomic booking function.
4. On success, the appointment is created and two notification outbox rows (confirmation to each party) plus scheduled reminder rows are written.
5. On conflict, the student sees slot taken and the slot list refreshes.

6.3 Messaging

1. Student sends a message via Server Action.
2. The action validates, checks the thread belongs to the student, and inserts the message.
3. RLS ensures only the student and the assigned counsellor can read it.
4. A new-message outbox entry (no content) is created for the recipient, and a realtime event updates an open thread.
5. Read status updates when the recipient opens the thread.

6.4 Mood check-in and crisis

1. Student submits a check-in. Data is stored with an optional safety flag.

2. If the student selected the explicit not-safe option, or taps Alert a counsellor, the crisis function runs.
3. The function creates a crisis event, finds the assigned or designated on-duty counsellor, and writes an urgent outbox entry.
4. The student immediately sees the helplines and the message that the counsellor *has been alerted* (FR-30).
5. If alert creation fails, the UI instructs the student to call a helpline now (PRD Section 14).
6. The counsellor sees the alert in the dashboard and records the outcome.

6.5 Notifications (outbox pattern)

1. Any feature that needs an email inserts a row into the notification outbox (type, recipient, related ID, send-at time).
2. A cron job picks up due rows, sends them via the email provider, and marks success or failure.
3. Failed rows retry with a limit, then surface in the platform admin overview.
4. Emails use fixed templates. They contain no message text, notes, or concern details (FR-43).

6.6 Admin analytics

1. Admin opens the dashboard.
2. The server calls the aggregate analytics function with the admin's institution.
3. The function returns counts and averages only, with groups smaller than 5 suppressed.
4. Export (FR-39) is generated from the same function output, never from raw tables.

6.7 Privacy requests

1. Student requests export or deletion from the Privacy page.
2. Re-authentication is required for deletion (NFR-6).
3. The privacy function assembles the export or applies deletion with the documented retention exception.
4. The action is written to the audit log.

7. Folder Structure

A single repository, a single Next.js app.

```
ilo/
├── README.md
├── PRD.md
├── ARCHITECTURE.md
├── .env.example                  # variable names only, no values
├── package.json
├── tsconfig.json
├── next.config
├── middleware                    # session refresh + route protection
│
├── supabase/
│   ├── migrations/               # ordered SQL migrations (schema, RLS,
functions)
│   ├── seed/                     # development seed data only
│   ├── tests/                    # SQL-level RLS and function tests
│   └── config                    # local Supabase configuration
│
├── src/
│   ├── app/
│   │   ├── (public)/             # landing, sign-in, verify, privacy notice
│   │   ├── (student)/            # S1-S11
│   │   ├── (counsellor)/         # C1-C8
│   │   ├── (admin)/              # A1-A4
│   │   ├── (platform)/           # PA1-PA2
│   │   ├── auth/                 # callback and sign-out route handlers
│   │   ├── api/
│   │   │   ├── cron/             # reminders, notification retry (secret-
protected)
│   │   │   ├── export/           # admin report export
│   │   │   └── health/           # uptime check, no data
│   │   ├── error / not-found / forbidden pages
│   │   └── layout
│   │
│   ├── features/                 # one folder per domain
│   │   ├── auth/
│   │   ├── consent/
│   │   ├── booking/
│   │   ├── messaging/
│   │   ├── checkin/
```

```
|   |   ├── crisis/
|   |   ├── notes/
|   |   ├── resources/
|   |   ├── admin/
|   |   └── platform/
|   |       (each contains: actions, schemas, service, data access,
components)
|   |
|   ├── components/              # shared UI primitives, layouts, skeletons,
states
|   ├── lib/
|   |   ├── supabase/            # user-scoped clients (browser and server)
|   |   ├── privileged/          # ONLY place allowed to use the service key
|   |   ├── email/               # provider interface + templates
|   |   ├── auth/                # session and role helpers
|   |   ├── errors/              # error types and result helpers
|   |   ├── logging/             # scrubbed logger
|   |   └── config/              # validated environment configuration
|   ├── types/                   # generated database types
|   └── styles/                  # theme tokens, global styles
|
├── tests/
|   ├── unit/
|   └── e2e/                     # Playwright journeys
|
└── .github/workflows/           # CI pipeline
```

Rules

- Each feature folder owns its actions, schemas, service logic, and data access. Cross-feature imports go through the feature's public entry file only.
- **Only `lib/privileged/` may use the service-role key.** A lint rule or CI check blocks imports of that folder from anywhere except designated server-only entry points (cron handlers, invitation actions, analytics, privacy functions).
- Nothing in `components/` or any client file may import server-only or privileged modules.

8. Database Interaction

8.1 Access pattern

- **Default:** the server creates a database client from the user's session. Every query runs as that user, so RLS applies automatically.
- **Privileged client:** used only for the paths listed in Section 10.3.
- **Generated types:** produced from the schema after each migration, so queries are type-checked.
- **No ORM and no raw string-built SQL.** Use the client's query builder or named database functions.

8.2 Schema ownership

- The schema is defined **only** through versioned migration files in `supabase/migrations/`. No manual dashboard edits in staging or production.
- Entities follow PRD Section 10.1: institutions, users (profiles), student and counsellor profiles, consent records, availability rules, appointments, message threads and messages, mood check-ins, session notes, crisis events, resources, notification outbox, audit log.
- Additional structural table: **counsellor-student assignment**, which defines who may see whose data. [AA2] Assignment is created when a student books with or messages a counsellor for the first time, and an Institution Admin can manage reassignment later.
- Every table carries the institution identifier so isolation (NFR-2) can be enforced uniformly.

8.3 Integrity rules (enforced by the database, not only the app)

- No overlapping appointments per counsellor.
- Appointment status transitions are limited to valid ones.
- Consent records are insert-only (immutable log).
- Audit log is insert-only.
- A student has one active thread per counsellor.
- Check-in scores fall within the allowed range.

8.4 Row-Level Security policy model

RLS is **on for every table**, with a default of deny. Policies are written against small helper functions: current user's role, current user's institution, and whether the current counsellor is assigned to a given student.

Data	Student	Counsellor	Institution Admin	Platform Admin
Own profile	Read/update own	Read/update own	Read/update own	Read/update own
Other users in institution	Counsellor directory fields only	Assigned students: preferred name only	Counsellors only (not students)	Institution-level metadata
Appointments	Own	Own (as counsellor)	**No row access**	**No row access**
Messages	Own threads	Own threads	**None**	**None**
Check-ins	Own	Assigned students **and** consent toggle on	**None**	**None**
Session notes	**None**	Own authored notes only	**None**	**None**
Crisis events	Own (status only)	Assigned or on-duty	**Counts via aggregate function only**	**None**
Consent records	Own (read)	None	None	None
Resources	Read (institution + global)	Read	Manage own institution's	Manage global
Analytics	None	None	Aggregate function, own institution	Aggregate function, all

This mirrors the PRD permission matrix (Section 5). **NFR-4 is satisfied by the absence of any policy granting admins access to messages or notes.**

8.5 Audit logging

- Database triggers or function-level writes record who accessed a student record, session note, or crisis event, and when. Content is never written to the audit log (PRD Section 10.2).
- Audit entries are readable only by Platform Admin through a restricted view for breach investigation, and contain no student content. [AA3]

8.6 Migrations and environments

- Migrations are applied through the Supabase CLI, first to staging, then to production, via the CI pipeline.
- Destructive migrations (dropping columns or tables) require manual approval.